



**UTM**  
UNIVERSITI TEKNOLOGI MALAYSIA



**CAIRO**  
UTM

# **JETSON AI WORKSHOP**

## **JETSON, GITHUB & PYTHON PROGRAMMING (PART 2)**

### **PREPARED BY:**

Center for Artificial Intelligence and Robotics  
Universiti Teknologi Malaysia

# Contents



1. What is GitHub?
2. Benefits of using GitHub
3. Understanding & Relationship of Git & GitHub
4. Setting up GitHub
5. Creating & Managing Repository
6. Understanding Branches
7. Issues in GitHub
8. Security on GitHub
9. Setting up GitHub in Jetson
10. Accessing Git
11. Jetson to and from GitHub
12. Conclusion

# WHAT IS GitHub?



# WHAT IS GitHub?



- Definition: GitHub is a web-based platform for version control and collaboration, built on top of Git.
- Purpose: It allows multiple people to work on a project simultaneously, track changes, and manage code repositories efficiently.
- Features: Hosting for Git repositories, collaboration tools, issue tracking, continuous integration and deployment (CI/CD) with GitHub Actions, and more.



# BENEFITS OF USING GitHub



## 1. Building a Professional Portfolio

- **Showcase Your Work:** Use GitHub repositories to showcase projects, demonstrating your coding skills and project management abilities to potential employers.
- **Contributions:** Highlight contributions to open-source projects, showing collaboration and teamwork skills.

## 2. Learning from Open-Source Projects

- **Access to Code:** Explore and learn from the code of experienced developers by browsing popular open-source repositories.
- **Contributing:** Improve your skills by contributing to open-source projects, which also helps build your reputation in the developer community.

# BENEFITS OF USING GitHub



## 3. Collaboration and Teamwork

- **Working in Teams:** GitHub is widely used in professional settings for collaborative work. Learning to use it effectively prepares you for teamwork in real-world projects.
- **Feedback and Code Reviews:** Engage in code reviews and discussions, receiving feedback and improving your code quality.

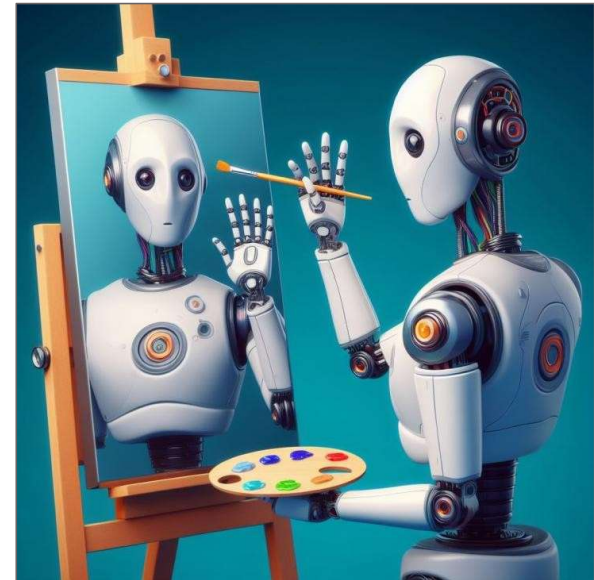
## 4. Project Management Skills

- **Issues and Project Boards:** Manage tasks, bugs, and project milestones using GitHub Issues and Project Boards, enhancing your project management skills.
- **Workflow Automation:** Use GitHub Actions to automate repetitive tasks, learning about CI/CD pipelines and improving productivity.

# BENEFITS OF USING GitHub

## 5. Continuous Learning

- **Staying Updated:** Follow repositories and developers to stay updated on the latest technologies and trends in the software development industry.
- **GitHub Education:** Take advantage of resources and tools provided by GitHub Education for students.



# UNDERSTANDING GIT & GitHub



## Git: A Version Control System

- Git is a distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

### Key Features

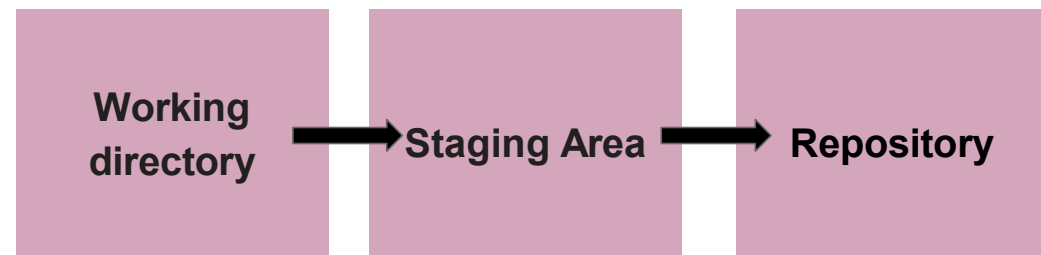
**Distributed:** Every developer has a full copy of the project history.

**Branching and Merging:** Support for nonlinear development through branches.

**Performance:** Optimized for high performance.

**Security:** Ensures the integrity of the code and history.

### Workflow



**Working Directory:** The files in your project.

**Staging Area:** A place to store changes before committing them.

**Repository:** The history of all commits and branches.



# UNDERSTANDING GIT & GitHub



**GitHub:** A Cloud-Based Hosting Service for Git Repositories

- GitHub is a platform for hosting and collaborating on Git repositories.



## **GitHub vs. Other Git Hosting Services:**

- GitHub: Known for its user-friendly interface, community engagement, and GitHub Actions for Continuous Integration (CI)/ Continuous Deployment/Delivery(CD).
- Alternatives: GitLab, Bitbucket, and others, each with unique features and strengths.

## **Key features**

1. Collaboration: Tools for code review, issue tracking, and project management.
2. Integration: Supports integration with CI/CD tools and other development tools.
3. Community: Millions of developers and repositories available for collaboration and learning.

# RELATIONSHIP OF GIT & GitHub



- **Git:**

**Tool:** Command-line tool installed locally on your computer.

**Function:** Version control system to track changes in your project.

- **GitHub:**

**Service:** Online platform that hosts Git repositories.

**Function:** Facilitates collaboration, code sharing, and project management.

- **Relationship:**

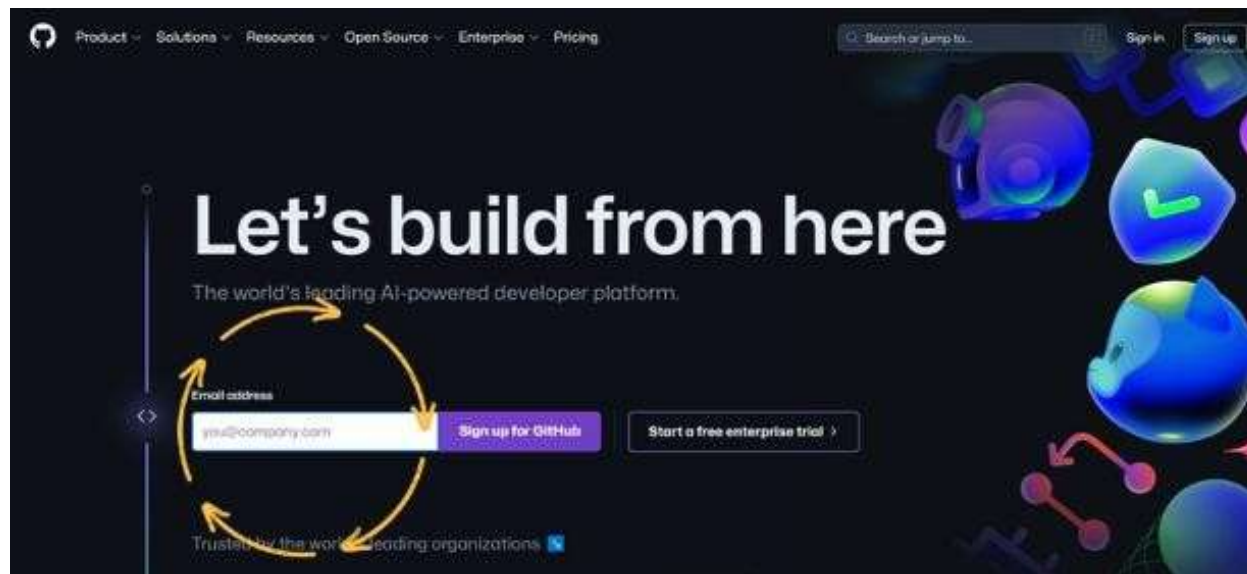
**Workflow:** Developers use Git to track changes locally and push/pull changes to/from GitHub for collaboration.

# SETTING UP GitHub



## Creating a GitHub Account:

**Sign Up:** Go to GitHub's website and enter your username, email, and password. Click "SIGN UP". Follow the prompts to complete the registration.



# SETTING UP GitHub



**Email Verification:** Verify your email address by clicking the link sent to your inbox.

This is the first screen of the GitHub account creation process. It has a dark blue background with white text. The text says "Welcome to GitHub! Let's begin the adventure." followed by three input fields: "Enter your email\*", "Create a password\*", and "Enter a username\*". The email field contains "muhammadthaqifiman@graduate.utm.my", the password field contains "\*\*\*\*\*", and the username field contains "Thaqifiman". A green "Continue" button is to the right of the username field. At the bottom, it says "Thaqifiman is available." and a small disclaimer about terms of service and privacy policy.This is the third screen of the GitHub account creation process. It has a dark blue background with white text. The text says "You're almost done!" followed by "We sent a launch code to muhammadthaqifiman@graduate.utm.my". Below this is a prompt "Enter code\*" followed by eight empty square boxes for entering the code.

# CREATING AND MANAGING REPOSITORIES



## 1. Creating a New Repository

Steps:

- Navigate to Repositories: Click on the "Create Repositories" tab in your GitHub page.

Fill in Details:

- Repository Name: Choose a unique name for your repository.
- Description: Optionally, add a description of your repository.
- Public/Private: Choose whether the repository will be public or private.
- Initialize with a README: Check this box to add a README file.
- Create Repository: Click the "Create repository" button.

# CREATING AND MANAGING REPOSITORIES



A screenshot of the GitHub Dashboard. The top navigation bar includes the GitHub logo, the word "Dashboard", a search bar with the placeholder "Type to search", and several utility icons. On the left sidebar, a section titled "Create your first project" has three orange arrows pointing to the "Create repository" button, the "Import repository" button, and the text "Ready to start building? Create a repository for a new idea or bring over an existing repository to keep contributing to it." The main content area is titled "Home" and contains several cards. The first card, "Start a new repository for LekoKeF", includes a text input for the repository name, radio buttons for "Public" and "Private" visibility, and a "Create a new repository" button. The second card, "Introduce yourself with a profile README", shows a preview of a README file named "LekoKeF / README.md" with a "Create" button. The third card, "Use tools of the trade", promotes GitHub Desktop. The fourth card, "Simplify your development workflow with a GUI", promotes GitHub Copilot. The right sidebar, titled "Explore repositories", lists several popular repositories: "pingcap / tidb", "lvgl / lvgl", and "armadaproject / armada", each with a star icon and a brief description. The URL "https://github.com/new" is visible in the bottom left corner of the screenshot.

# CREATING AND MANAGING REPOSITORIES



## Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (\*).

Owner \*

Repository name \*

Lekekef

/ testingggg

testingggg is available

Great repository names are short and memorable. Need inspiration? How about [symmetrical-octo-succotash](#)?

Description (optional)

This repo is for testing purpose onlyS

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

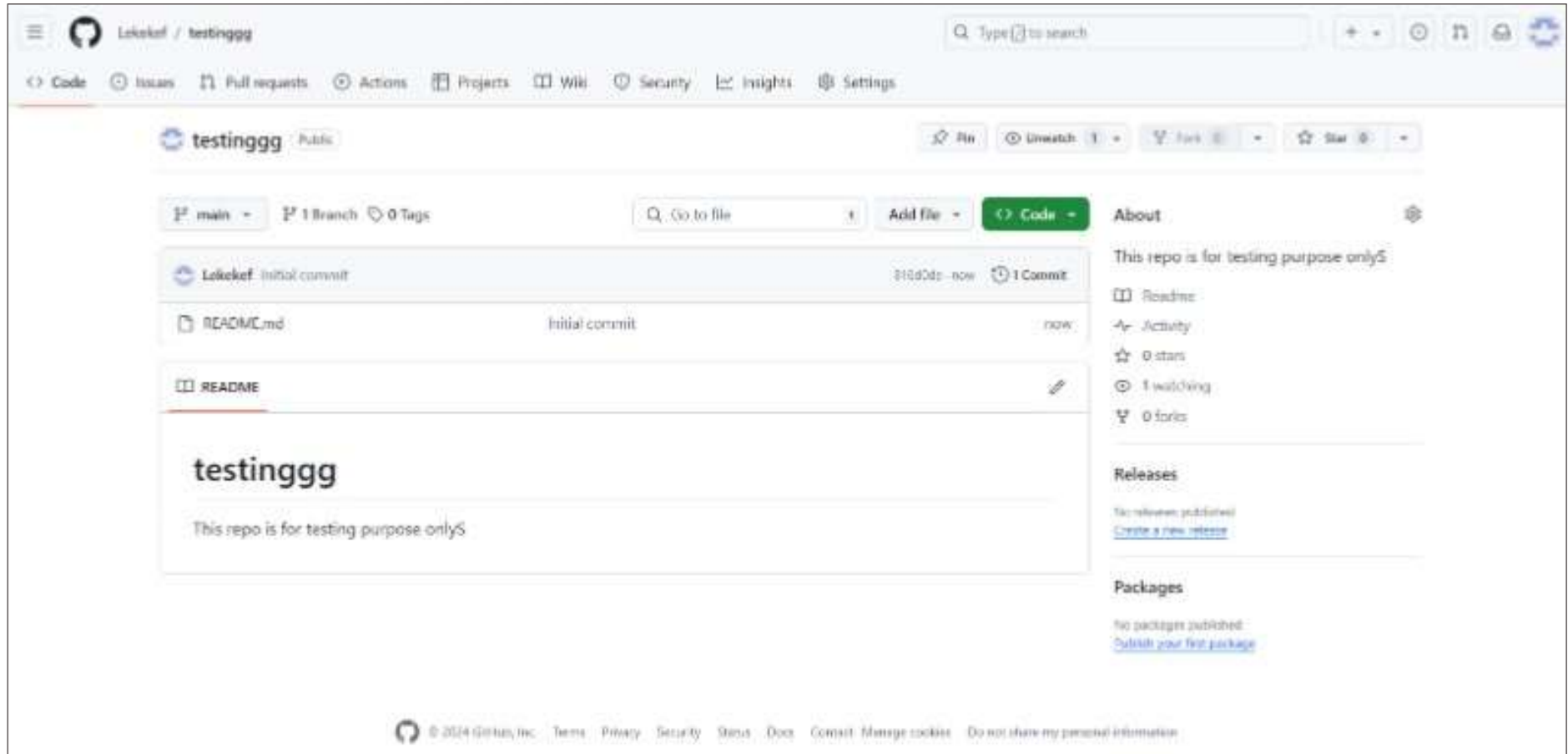
A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

**You are creating a public repository in your personal account.**

Create repository

# CREATING AND MANAGING REPOSITORIES





# CREATING AND MANAGING REPOSITORIES



## 2. README File

**Purpose:** A README file provides information about your project, such as what it does, how to use it, and how to contribute.

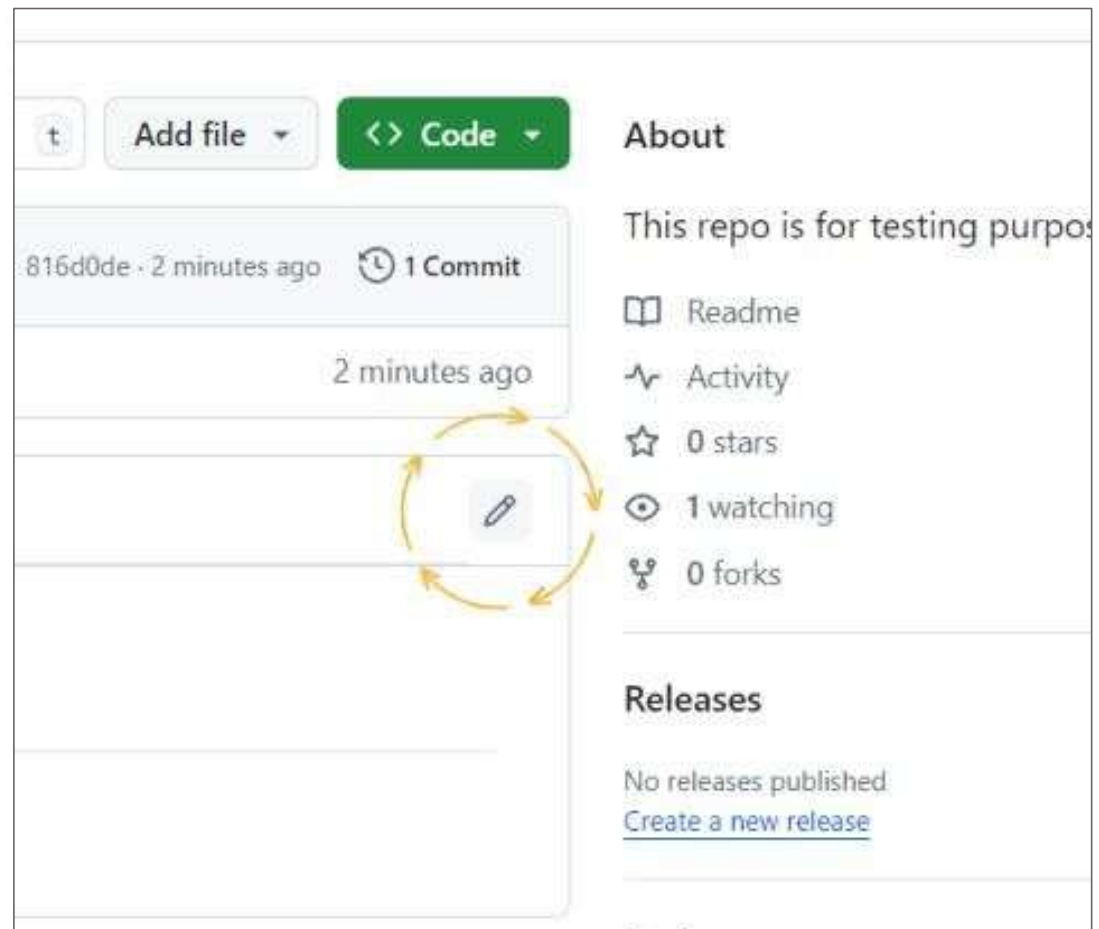
### Steps:

1. Navigate to Repository: Go to your newly created repository.
2. Add README: If you didn't initialize with a README, click "Add a README" on the repository home page.
3. Edit README: Use the GitHub editor to write and format your README in Markdown.
4. Commit Changes: Click "Commit changes" to save your changes.

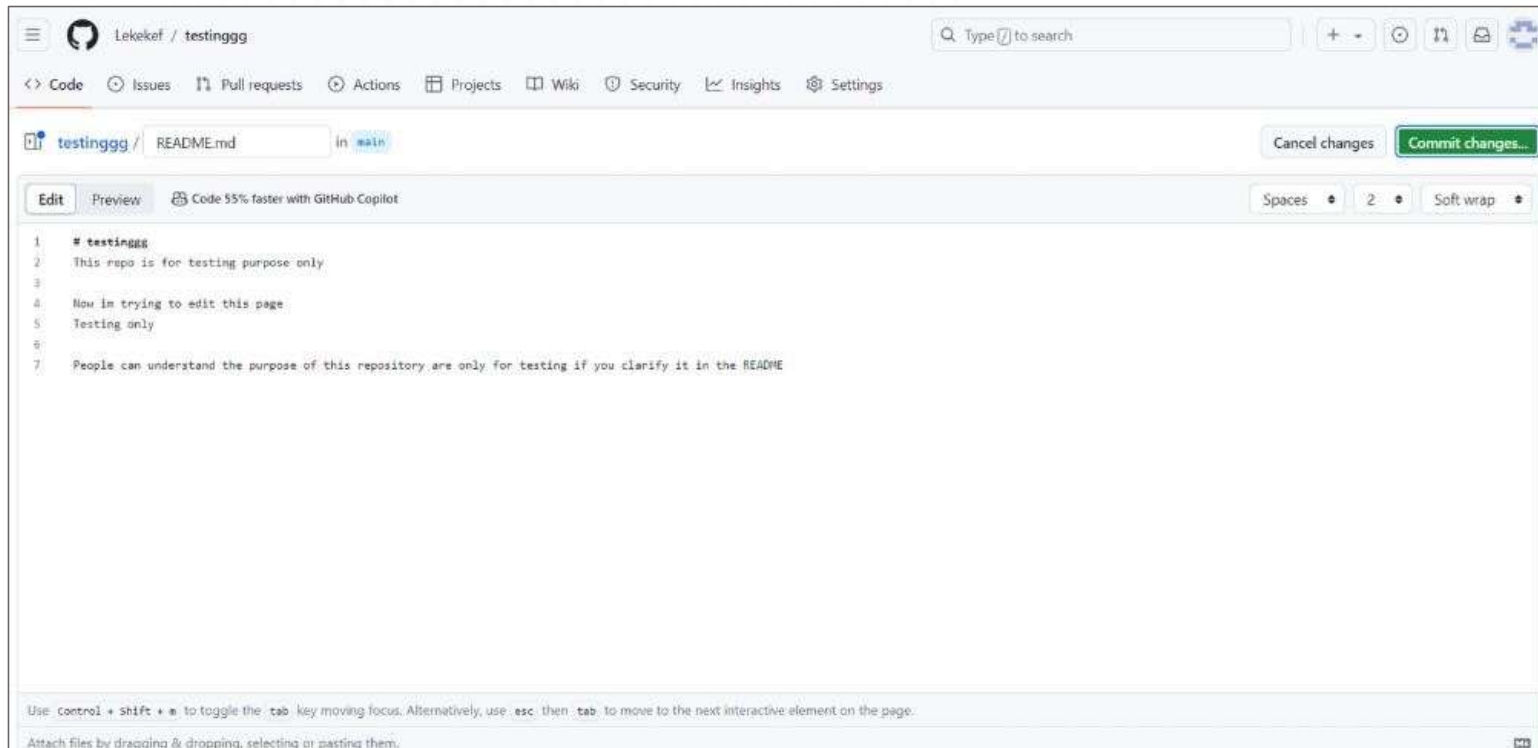
# CREATING AND MANAGING REPOSITORIES



**EDIT BUTTON HERE**  
click this to edit your  
**README**

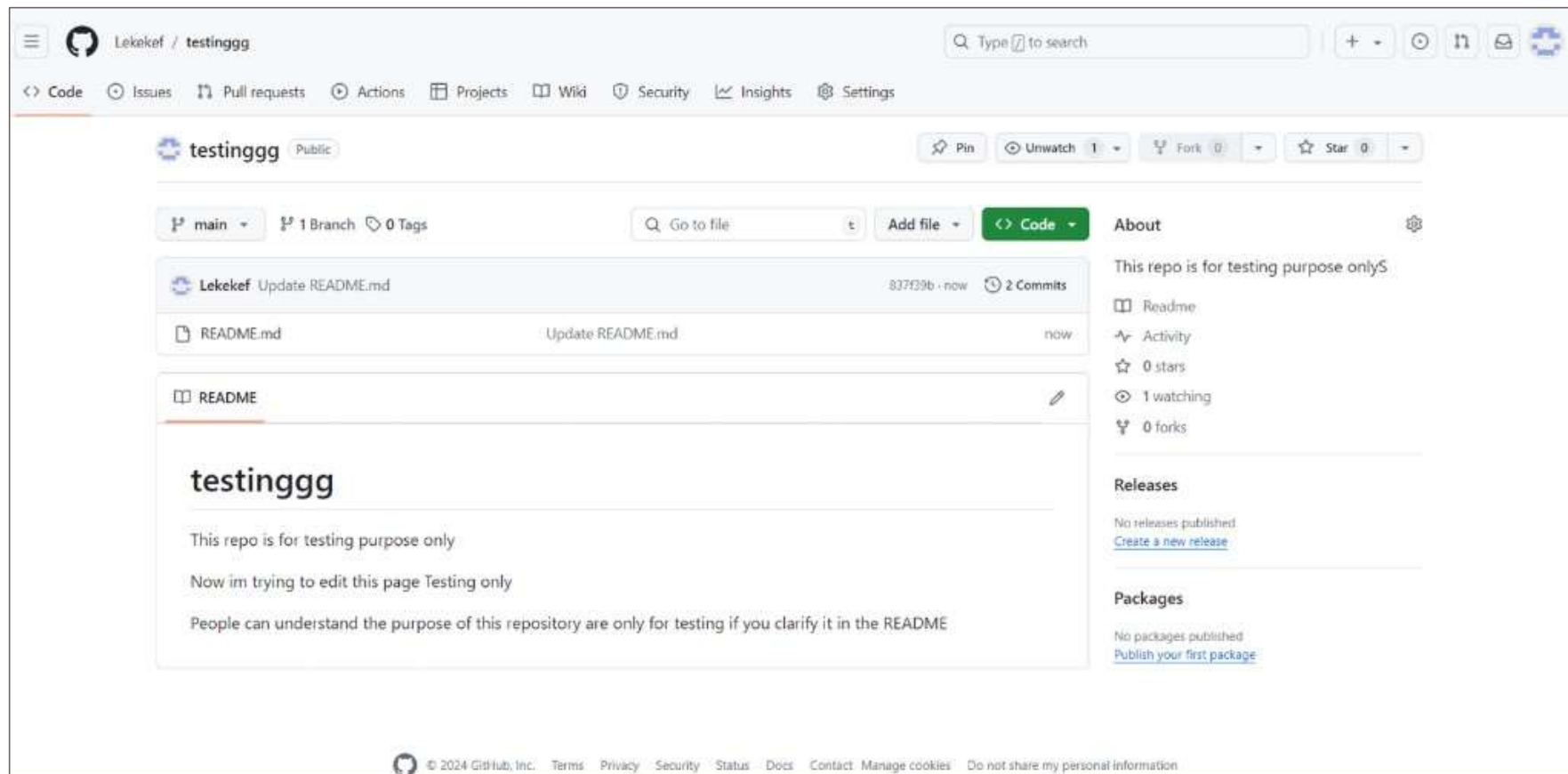


# CREATING AND MANAGING REPOSITORIES



**You may try and edit your README to give clear descriptions of your projects.  
Once done, click “Commit Changes” to save your edits.**

# CREATING AND MANAGING REPOSITORIES

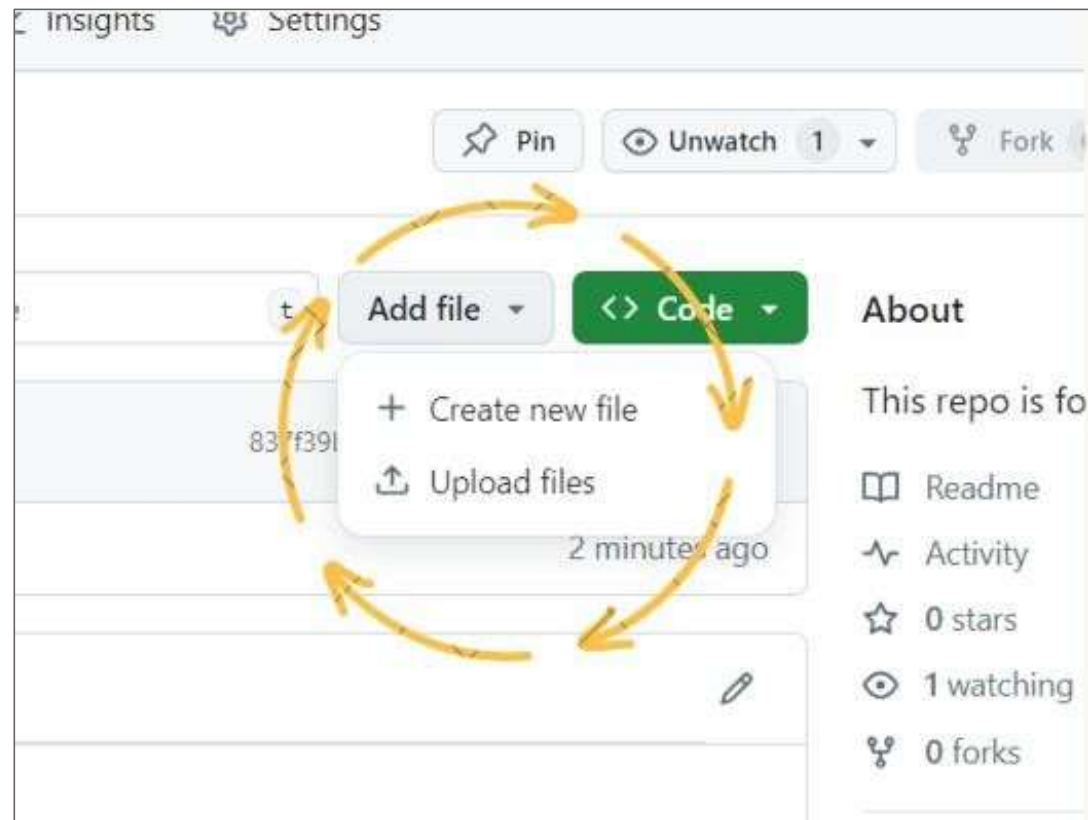


**You can view the changes on the main page**

# CREATING AND MANAGING REPOSITORIES



**Click here to try and  
UPLOAD FILES**



# CREATING AND MANAGING REPOSITORIES



The screenshot shows the GitHub web interface for a repository named "testinggg" by user "Lekekef". The file "requirements.txt" is being edited on the "main" branch. The file content is as follows:

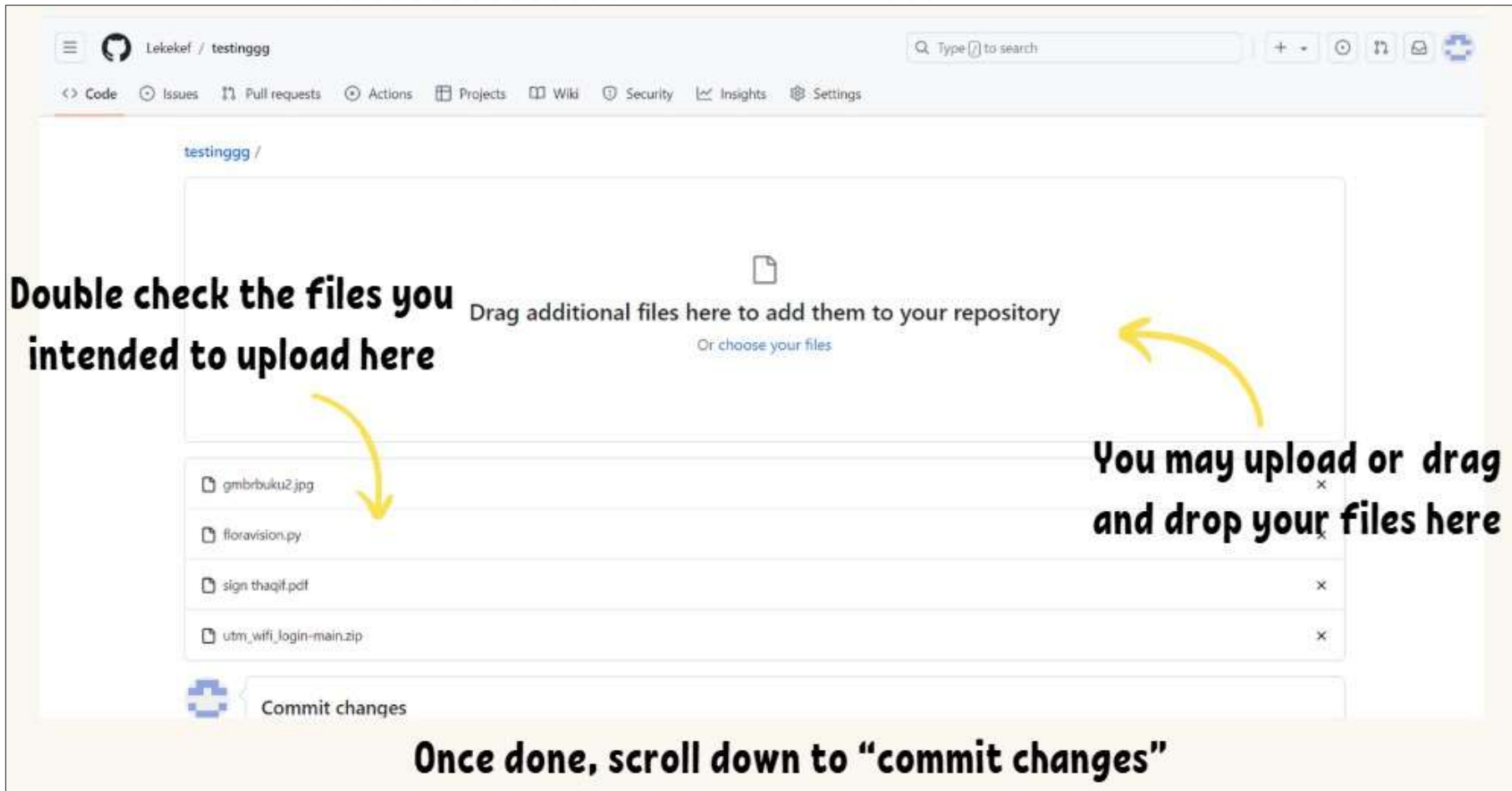
```
1 numpy
2 os
3 matplotlib
4 torchvision
5 cv2
```

Three yellow arrows and text annotations highlight key steps in the process:

- 1. Name your file**: An arrow points to the file name "requirements.txt" in the header.
- 2. Edit text into the file**: An arrow points to the code editor area where the file content is displayed.
- 3. "Commit changes" to save your file**: An arrow points to the green "Commit changes..." button in the top right corner.

Other visible elements include the GitHub navigation bar with links to Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. The editor interface includes tabs for "Edit" and "Preview", a GitHub Copilot suggestion, and formatting options like "Spaces" and "No wrap".

# CREATING AND MANAGING REPOSITORIES



The screenshot shows the GitHub interface for a repository named 'testingggg'. The main area is a large box with a document icon and the text 'Drag additional files here to add them to your repository' and 'Or choose your files'. Below this is a list of four files: 'gmbrbuku2.jpg', 'floravision.py', 'sign thaqif.pdf', and 'utm\_wifi\_login-main.zip'. At the bottom is a 'Commit changes' button. Annotations include: 'Double check the files you intended to upload here' with an arrow pointing to the file list; 'You may upload or drag and drop your files here' with an arrow pointing to the main upload area; and 'Once done, scroll down to "commit changes"' at the bottom.

testingggg /

Drag additional files here to add them to your repository  
Or choose your files

Double check the files you intended to upload here

You may upload or drag and drop your files here

gmbrbuku2.jpg  
floravision.py  
sign thaqif.pdf  
utm\_wifi\_login-main.zip

Commit changes

Once done, scroll down to "commit changes"

# UNDERSTANDING BRANCHES

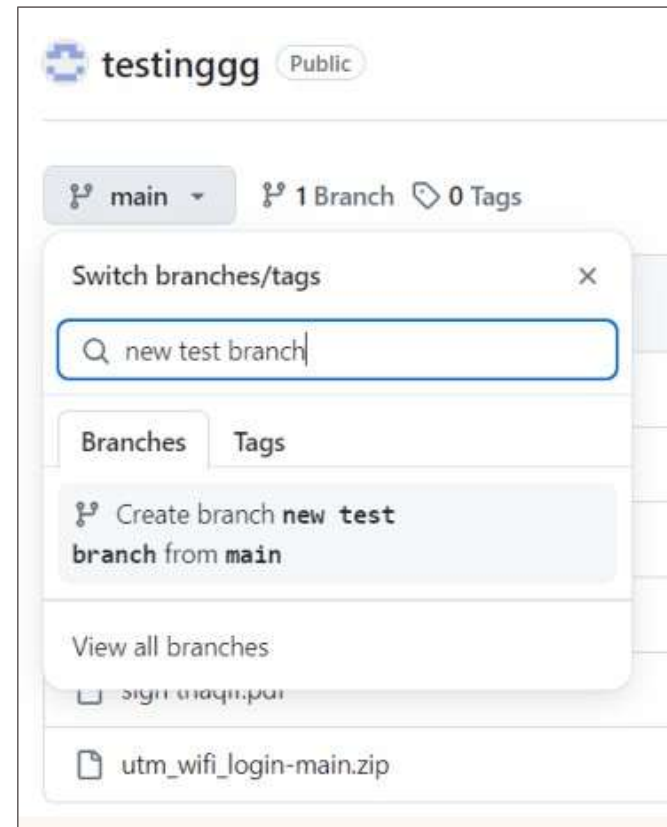


## 1. Creating Branches

**Purpose:** Branches allow you to develop features, fix bugs, or safely experiment with new ideas in a contained area.

### Steps:

- Navigate to Repository: Go to your repository on GitHub.
- Branch Menu: Click the branch dropdown menu (usually shows "main" or "master").
- Create Branch:  
Type a new branch name into the text box.  
Press "Enter" to create the new branch.





# UNDERSTANDING BRANCHES

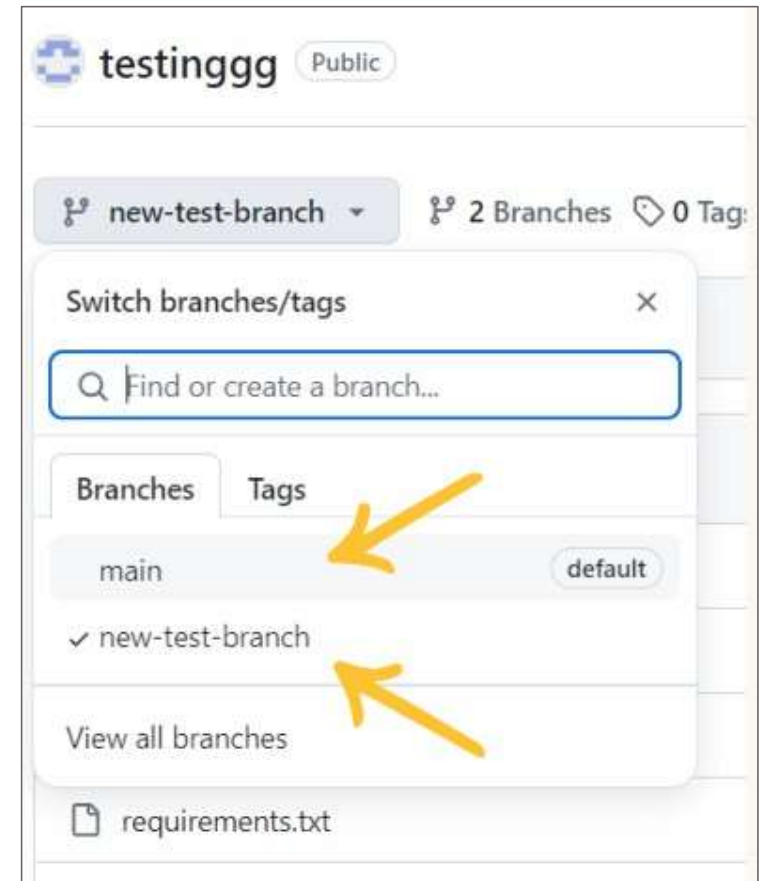


## 2. Switching Between Branches

**Purpose:** Switch between branches to view or work on different versions of your project.

**Steps:**

- Branch Menu: Click the branch dropdown menu.
- Select Branch: Click on the branch you want to switch to from the list.



# UNDERSTANDING BRANCHES



## 3. Comparing Branches

**Purpose:** Review the differences between branches to understand what changes have been made.

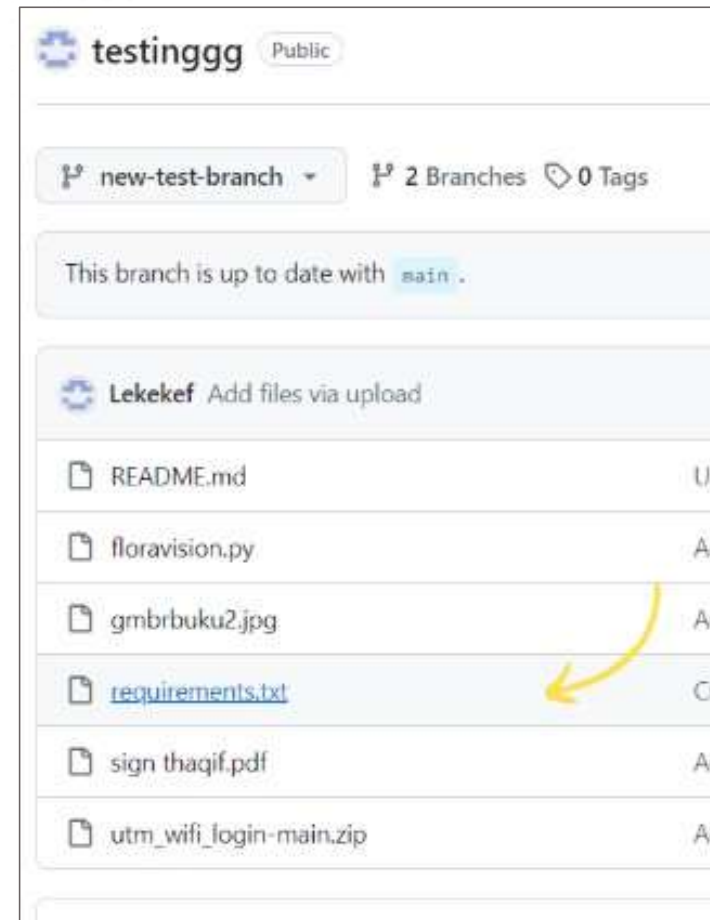
**Steps:**

- Navigate to Repository: Go to your repository on GitHub.
- Compare Button: Click the "Compare" button or navigate to the "Pull requests" tab and click "New pull request".
- Select Branches: Choose the base branch and compare branch.
- View Changes: GitHub will show a diff of the changes between the two branches.

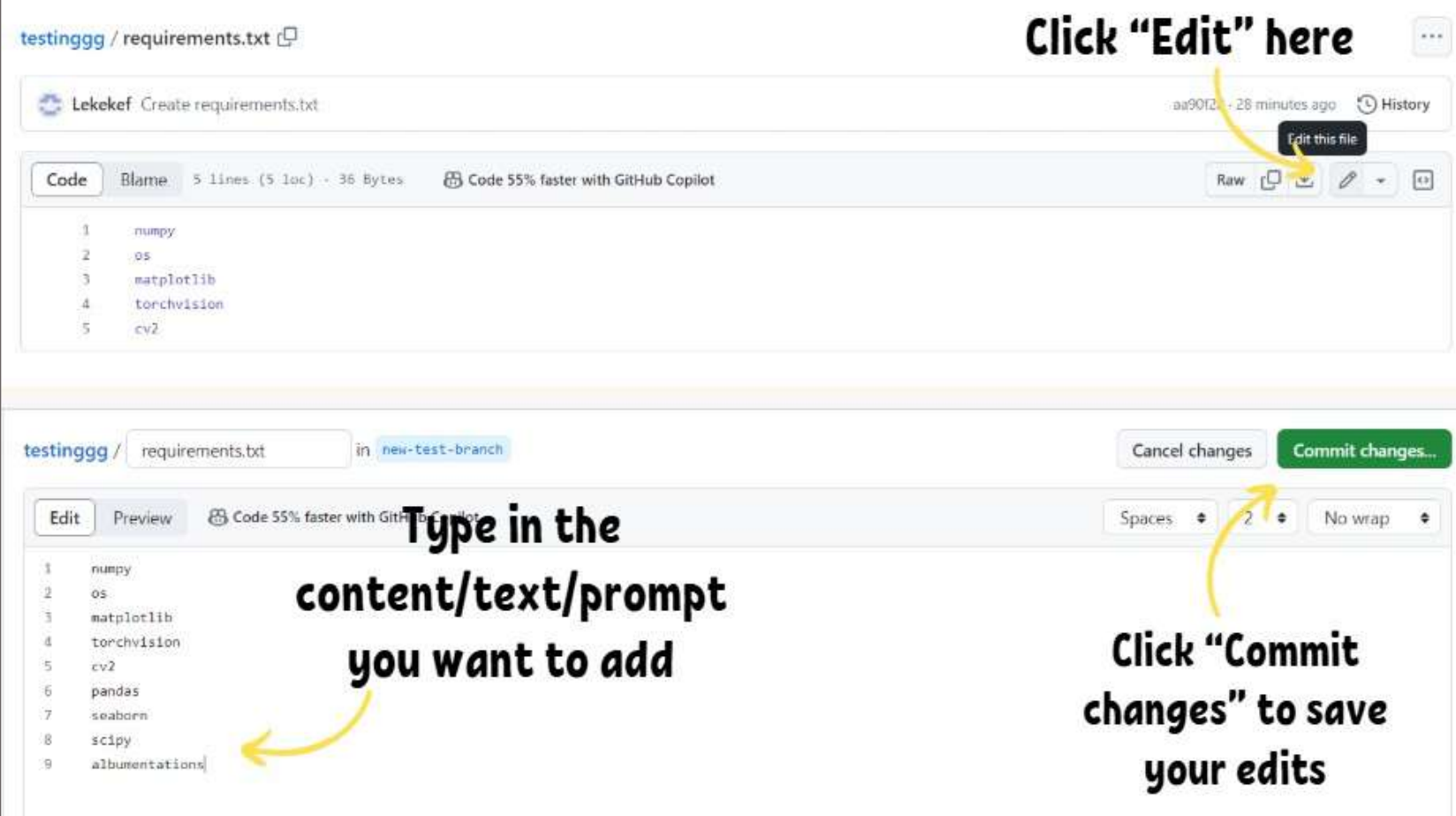
# UNDERSTANDING BRANCHES



**For example here, lets try and edit the content of this file**



# UNDERSTANDING BRANCHES



The screenshot displays the GitHub web interface for editing a file named `requirements.txt` in the `testinggg` repository. The top section shows the file's history, with the latest commit by `Lekekef` at `aa9012` from 28 minutes ago. The file content is shown with line numbers 1 to 5, listing dependencies: `numpy`, `os`, `matplotlib`, `torchvision`, and `cv2`. A yellow arrow points to the `Edit this file` button in the top right corner, with the text **Click "Edit" here**.

The bottom section shows the editing interface in the `new-test-branch`. The file content is updated to include `pandas`, `seaborn`, `scipy`, and `albumentations` on lines 6 through 9. A yellow arrow points to the text area, with the text **Type in the content/text/prompt you want to add**. The interface includes `Cancel changes` and `Commit changes...` buttons. A yellow arrow points to the `Commit changes...` button, with the text **Click "Commit changes" to save your edits**. The bottom right of the interface shows settings for `Spaces` (set to 2) and `No wrap`.

# UNDERSTANDING BRANCHES



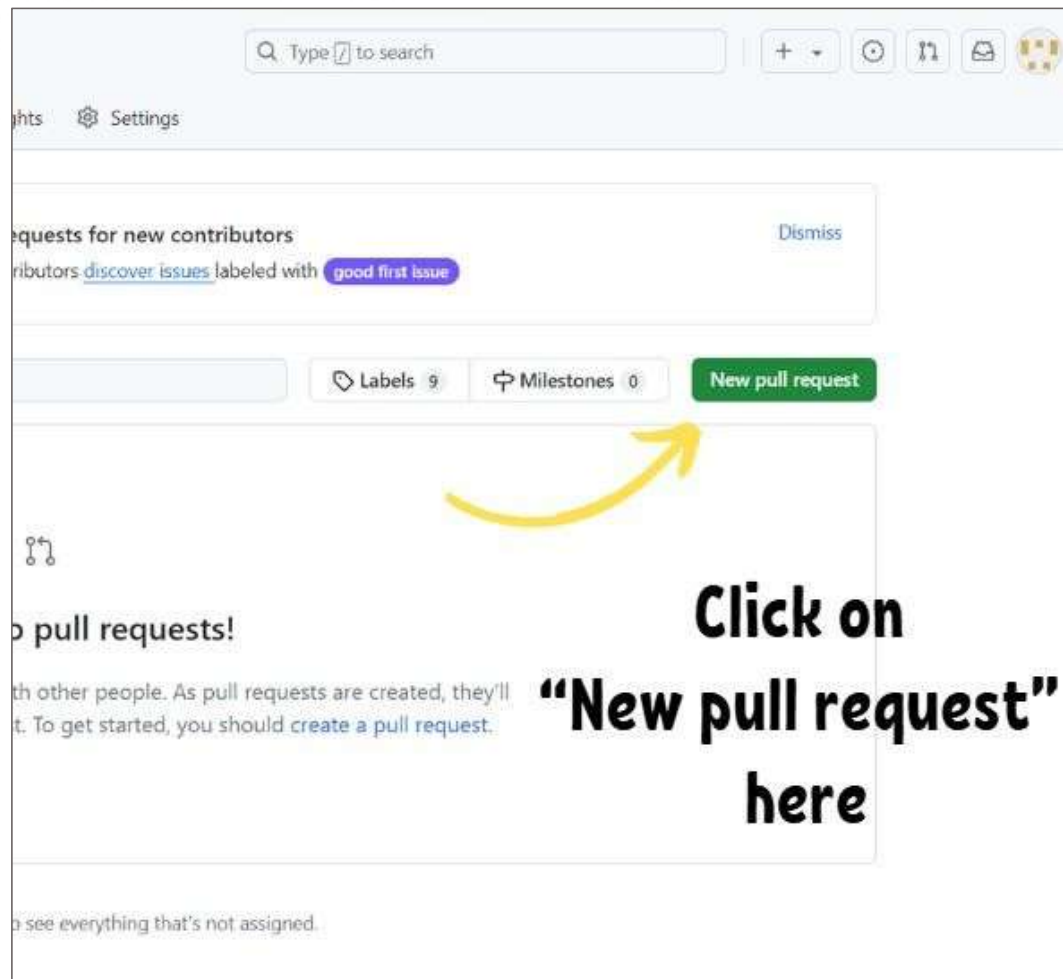
To compare the edited file back with the original file, click on the “Pull Requests” tab

The screenshot shows the GitHub interface for the repository 'Lekekef / testingggg'. A yellow arrow points to the 'Pull requests' tab in the top navigation bar. The repository page displays the 'new-test-branch' which is 1 commit ahead of 'main'. A yellow banner indicates recent pushes to 'new-test-branch' with a 'Compare & pull request' button. Below this, a table lists recent commits:

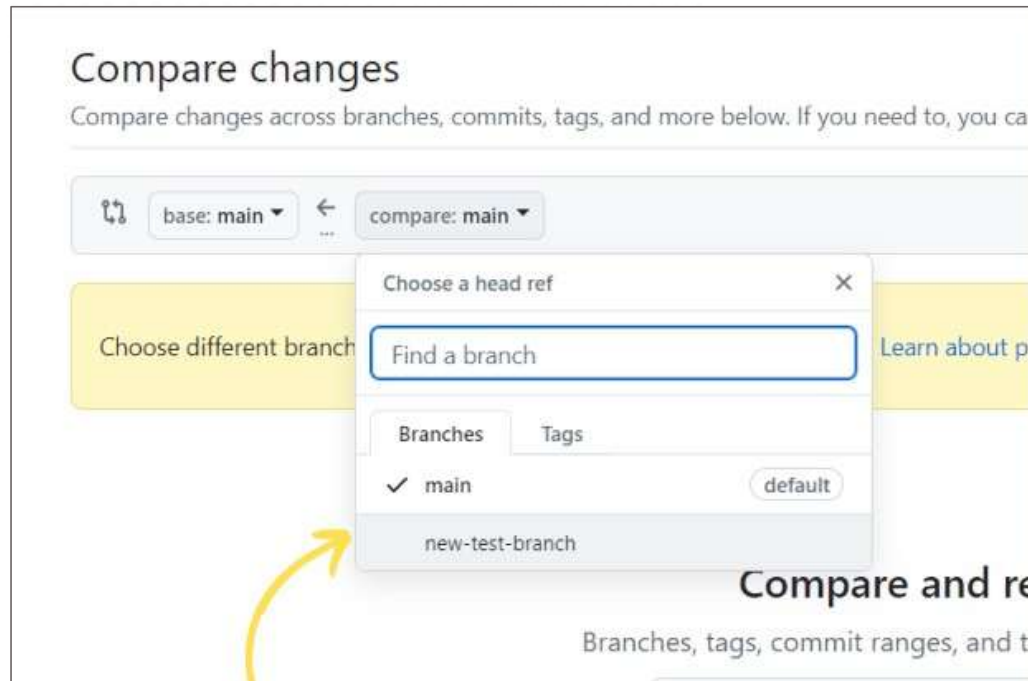
| Commit Hash | Commit Message          | Time Ago       |
|-------------|-------------------------|----------------|
| f639500     | Update requirements.txt | 1 minute ago   |
|             | Update README.md        | 49 minutes ago |
|             | Add files via upload    | 22 minutes ago |
|             | Add files via upload    | 22 minutes ago |

The right sidebar shows repository statistics: 0 stars, 1 watching, 0 forks, and a note that the repo is for testing purposes only.

# UNDERSTANDING BRANCHES



# UNDERSTANDING BRANCHES



**Next, we are going to compare the ‘edited file’ in the ‘new test branch’ with the ‘original file’ in the ‘main branch’. Choose the ‘new test branch’ here**

# UNDERSTANDING BRANCHES: EXAMPLE



Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: main ← compare: new-test-branch ✓ Able to merge. These branches can be automatically merged.

Update requirements.txt #1  
No description available [View pull request](#)

1 commit 1 file changed 1 contributor

Commits on Aug 8, 2024

Update requirements.txt  
Lekekef committed 1 hour ago Verified #639500

Showing 1 changed file with 4 additions and 0 deletions. Split Unified

requirements.txt

```
@@ -3,3 +3,7 @@ os
3 3 matplotlib
4 4 torchvision
5 5 cv2
6 + pandas
7 + seaborn
8 + scipy
9 + albumentations
```

**Here, we may view the new input done in the 'new test branch' unified with the original one in the 'main branch'**

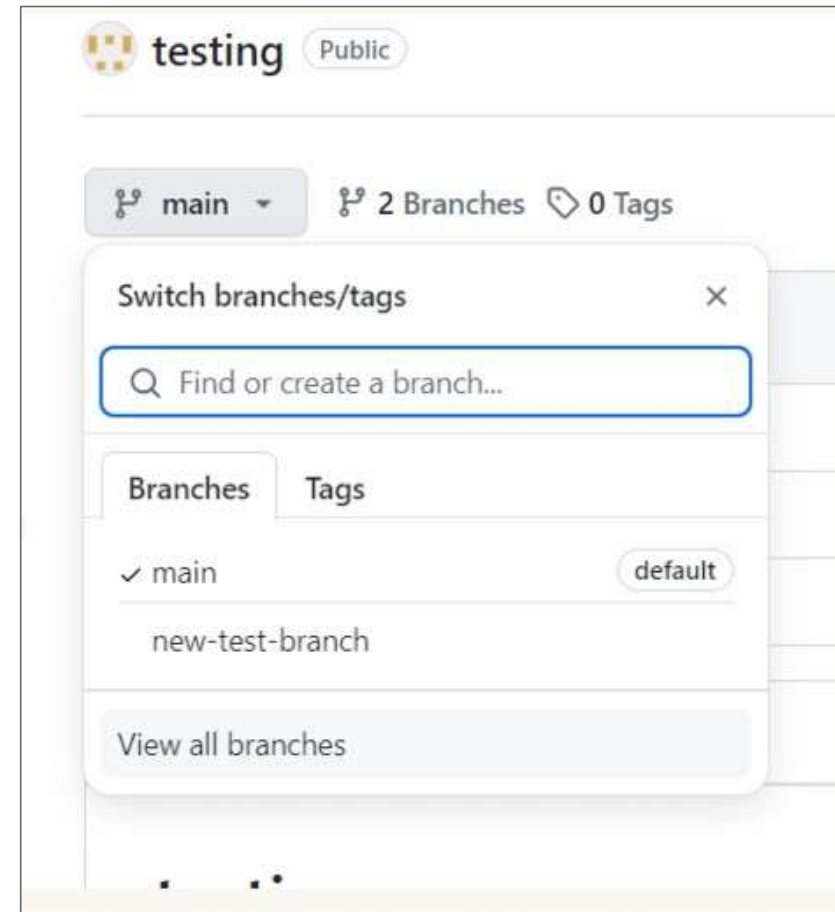


# UNDERSTANDING BRANCHES: DELETE BRANCH



## 4. Deleting Branches

- **Purpose:** Clean up unnecessary branches to keep repository organized
- **Step:**
  - Navigate to Repository: Go to your repository on GitHub.
  - Branches Tab: Click the "Branches" tab.
  - Delete Branch: Click the trash can icon next to the branch you want to delete. Note: You cannot delete the branch you are currently on.



# UNDERSTANDING BRANCHES: DELETE BRANCH



Overview | Yours | Active | Stale | All

Q Search branches...

**Default**

| Branch | Updated    | Check status | Behind | Ahead | Pull request |
|--------|------------|--------------|--------|-------|--------------|
| main   | 1 hour ago |              |        |       | Default  ... |

Your main branch isn't protected  
Protect this branch from force pushing or deletion, or require status checks before merging. [Learn more](#) Dismiss Protect this branch

**Your branches**

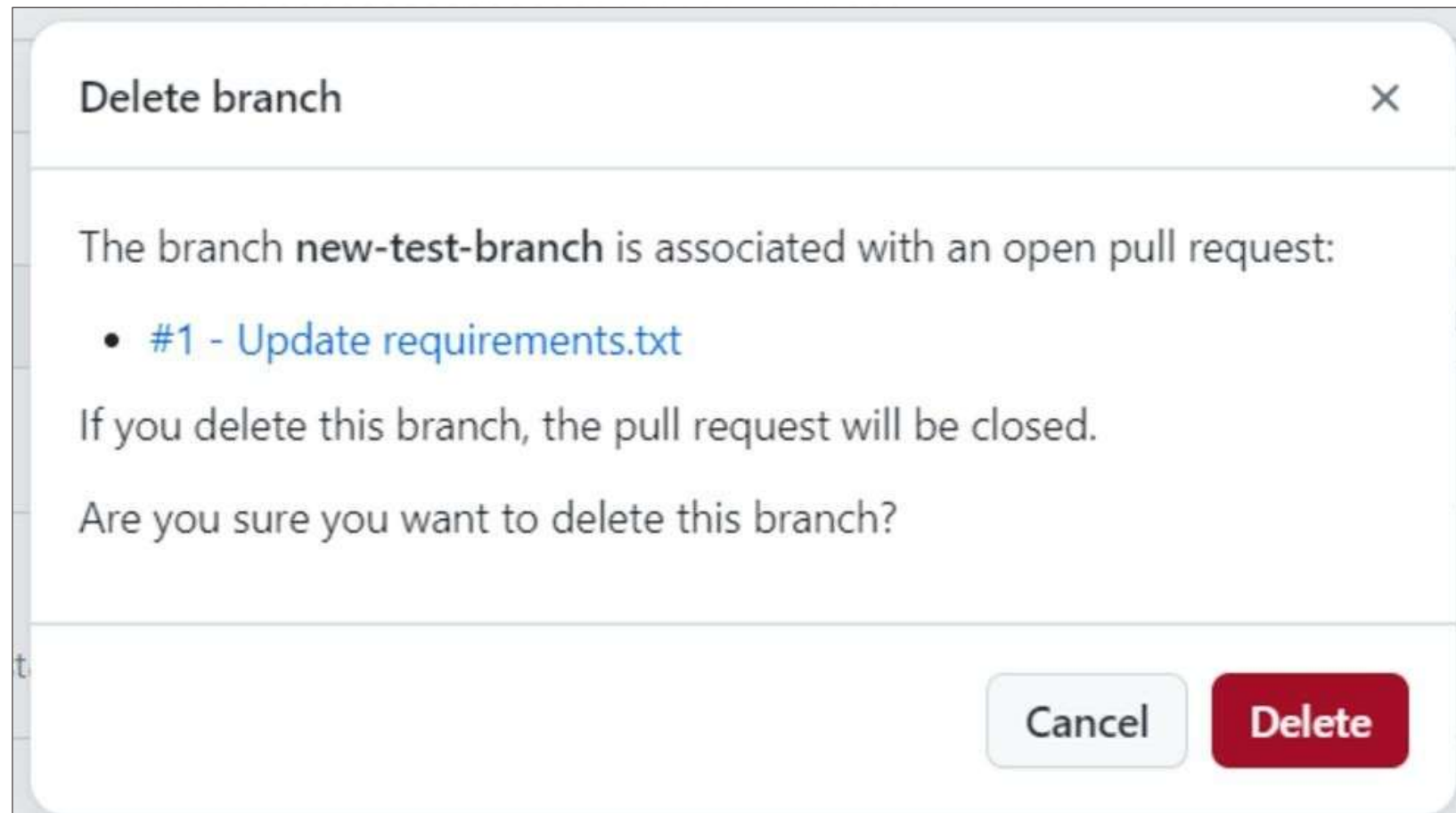
| Branch          | Updated    | Check status | Behind | Ahead | Pull request |
|-----------------|------------|--------------|--------|-------|--------------|
| new-test-branch | 1 hour ago |              | 0      | 1     | #1  ...      |

**Active branches**

| Branch          | Updated    | Check status | Behind | Ahead | Pull request |
|-----------------|------------|--------------|--------|-------|--------------|
| new-test-branch | 1 hour ago |              | 0      | 1     | #1  ...      |

**Click on the 'trash can' icon to delete your branch**

# UNDERSTANDING BRANCHES: DELETE BRANCH



# ISSUES



## 1. Creating and Managing Issues

- **Purpose:** Issues are used to track tasks, enhancements, and bugs in your project.
- **Steps:**
  - Navigate to Issues: Go to your repository and click the "Issues" tab.
  - New Issue: Click the "New issue" button.
  - Fill in Details:
    - Title: Provide a concise title for the issue.
    - Description: Describe the issue, including steps to reproduce, expected behavior, and screenshots if applicable.
- **Submit Issue:** Click "Create" to create the issue.

# ISSUES



## 2. Using Labels, Milestones, and Assignees

- **Labels:**
  - Navigate to Issues: Go to the "Issues" tab.
  - Select Issue: Click on an issue to view it.
  - Add Labels: Click the "Labels" button and select or create labels to categorize the issue.
- **Milestones:**
  - Navigate to Issues: Go to the "Milestones" tab within Issues.
  - New Milestone: Click "New milestone", add a title and description, and set a due date if needed.
  - Add Issues to Milestone: Open an issue, click "Milestone", and select the milestone to associate with the issue.
- **Assignees:**
  - Open Issue: Open the issue you want to assign.
  - Assign: Click the "Assignees" button and select the team member to assign the issue to.

# ISSUES: EXAMPLE



kekef / testinggg

Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

Filters  Labels 9 Milestones 0 [New issue](#)

☐ 0 Open ☒ 0 Closed

Author Label Projects Milestones Assignee Sort

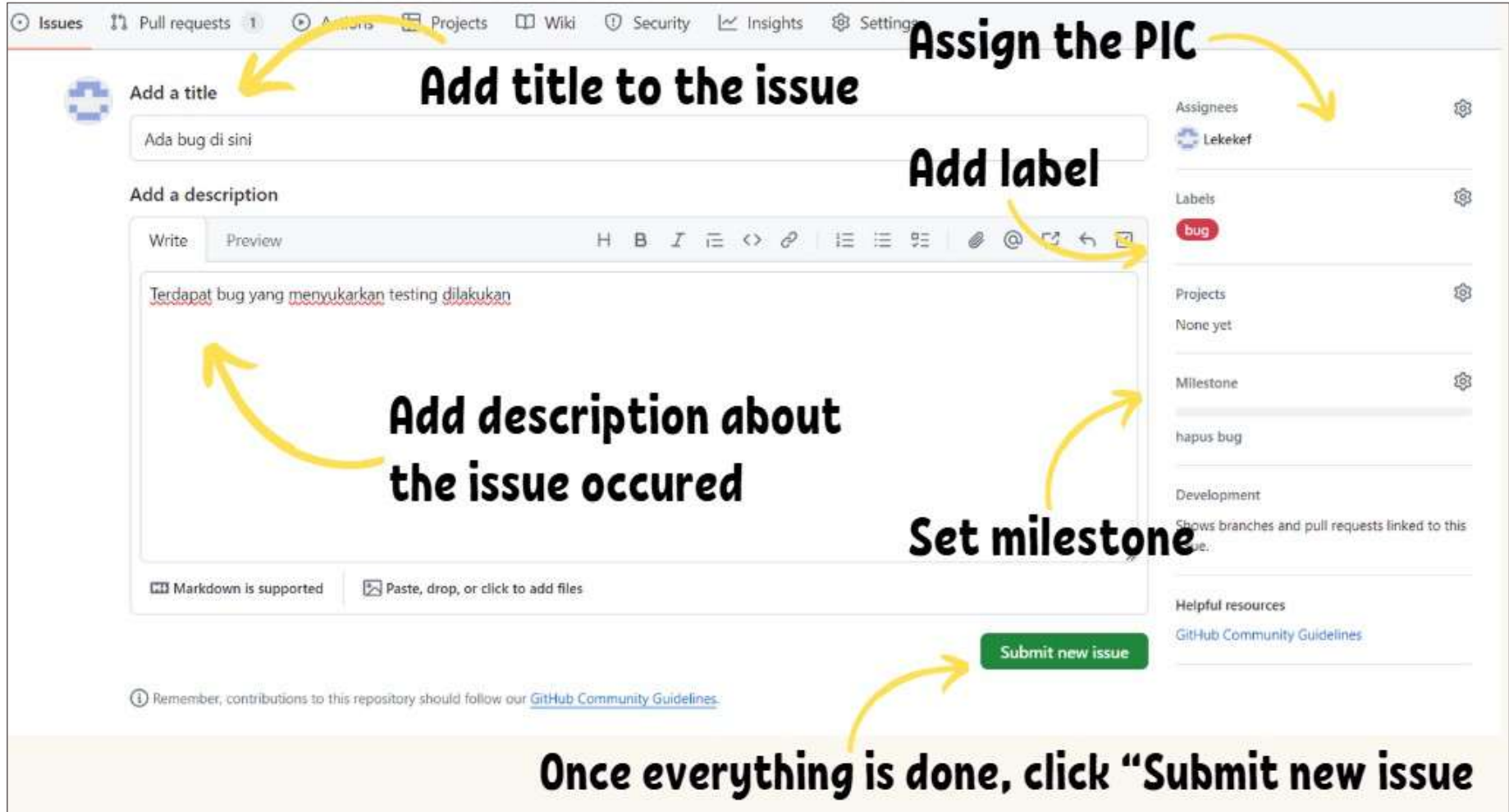
**Click on “New issue” to began**

There aren't any open issues.

You could search [all of GitHub](#) or try an [advanced search](#).

**ProTip!** Exclude everything labeled `bug` with `-label:bug`.

# ISSUES: EXAMPLE



The screenshot shows the GitHub 'New Issue' form with several annotations:

- Add title to the issue**: Points to the 'Add a title' field, which contains the text 'Ada bug di sini'.
- Add label**: Points to the 'Labels' section on the right, where a red 'bug' label is selected.
- Assign the PIC**: Points to the 'Assignees' section on the right, where 'Lekekef' is selected.
- Add description about the issue occurred**: Points to the 'Add a description' text area, which contains the text 'Terdapat bug yang menyukarkan testing dilakukan'.
- Set milestone**: Points to the 'Milestone' section on the right, where 'hapus bug' is selected.
- Once everything is done, click "Submit new issue"**: Points to the green 'Submit new issue' button at the bottom right.

The form also includes a 'Pull requests' tab, a 'Wiki' link, and a 'Security' link. The 'Submit new issue' button is located at the bottom right of the form.



# ISSUES: EXAMPLE

Ada bug di sini #2 **The issue has been successfully posted** New issue

Open Lekekef opened this issue now · 0 comments

**Lekekef commented now** Owner ...

Terdapat bug yang menyukarkan testing dilakukan

**Lekekef added the bug label now**

**Lekekef added this to the hapus bug milestone now**

**Lekekef self-assigned this now**

**People may add comments to help solve the issue**

**Add a comment**

Write Preview H B I          

Add your comment here...

 Markdown is supported  Paste, drop, or click to add files

Close issue Comment

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

**Assignees**  
 Lekekef

**Labels**  
bug

**Projects**  
None yet

**Milestone**  
hapus bug

**Development** Create a branch for this issue or link a pull request.

**Notifications** Customize  
Unsubscribe  
You're receiving notifications because you're watching this repository.

**1 participant**  


Lock conversation



# SECURITY ON GitHub

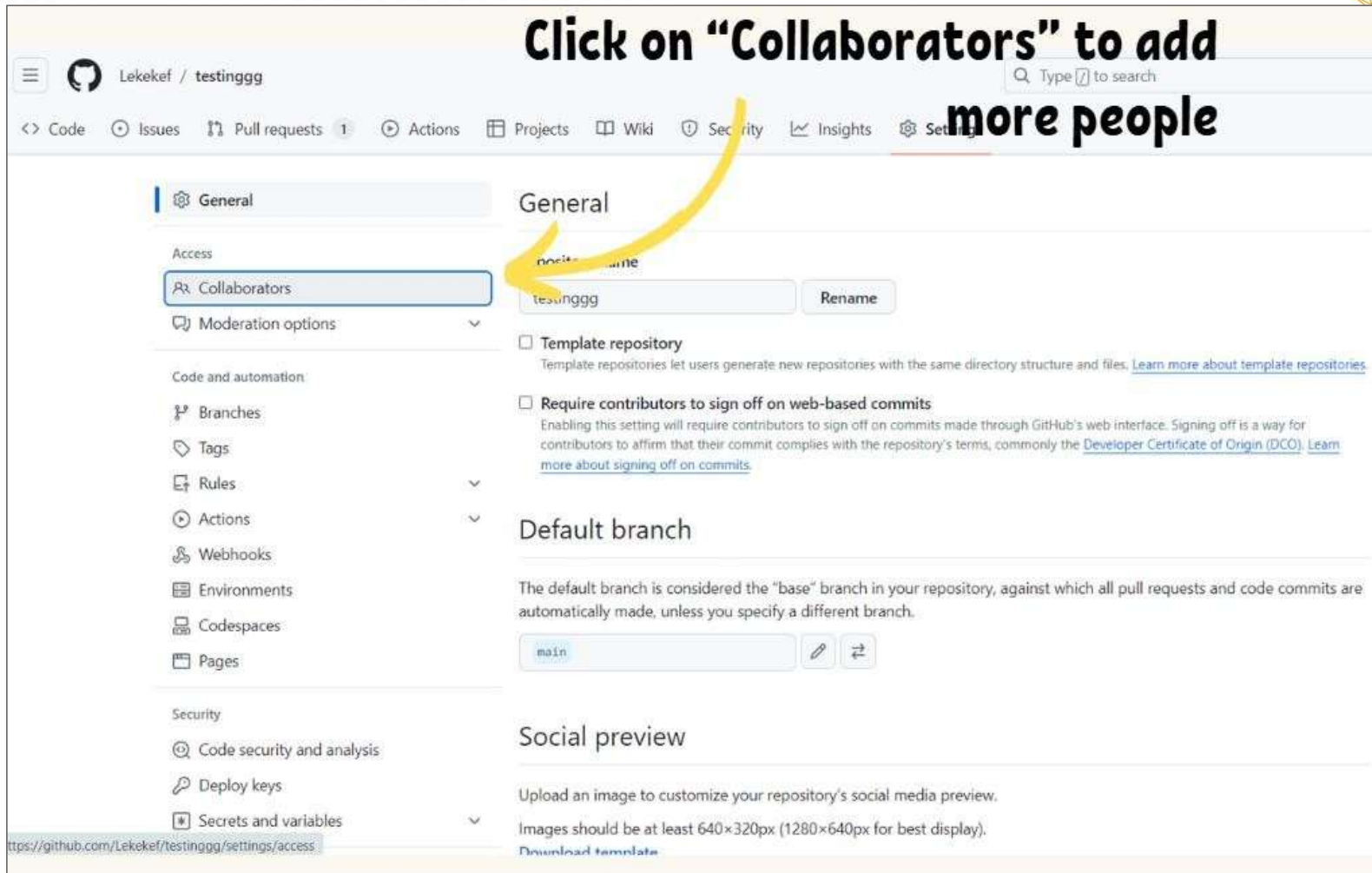


## Managing Access Control and Permissions

- **Purpose:** Control who can view and contribute to your repository.
- **Steps:**
  - Navigate to Settings: Go to your repository and click the "Settings" tab.
  - Manage Access: Click "Manage access".
  - Add Collaborators: Click "Invite a collaborator" to add users and set their permissions (Read, Write, Admin).

# SECURITY ON GitHub: EXAMPLE

**Click on "Collaborators" to add more people**



Lekekef / testinggg

Code Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Code security and analysis

Deploy keys

Secrets and variables

Template repository

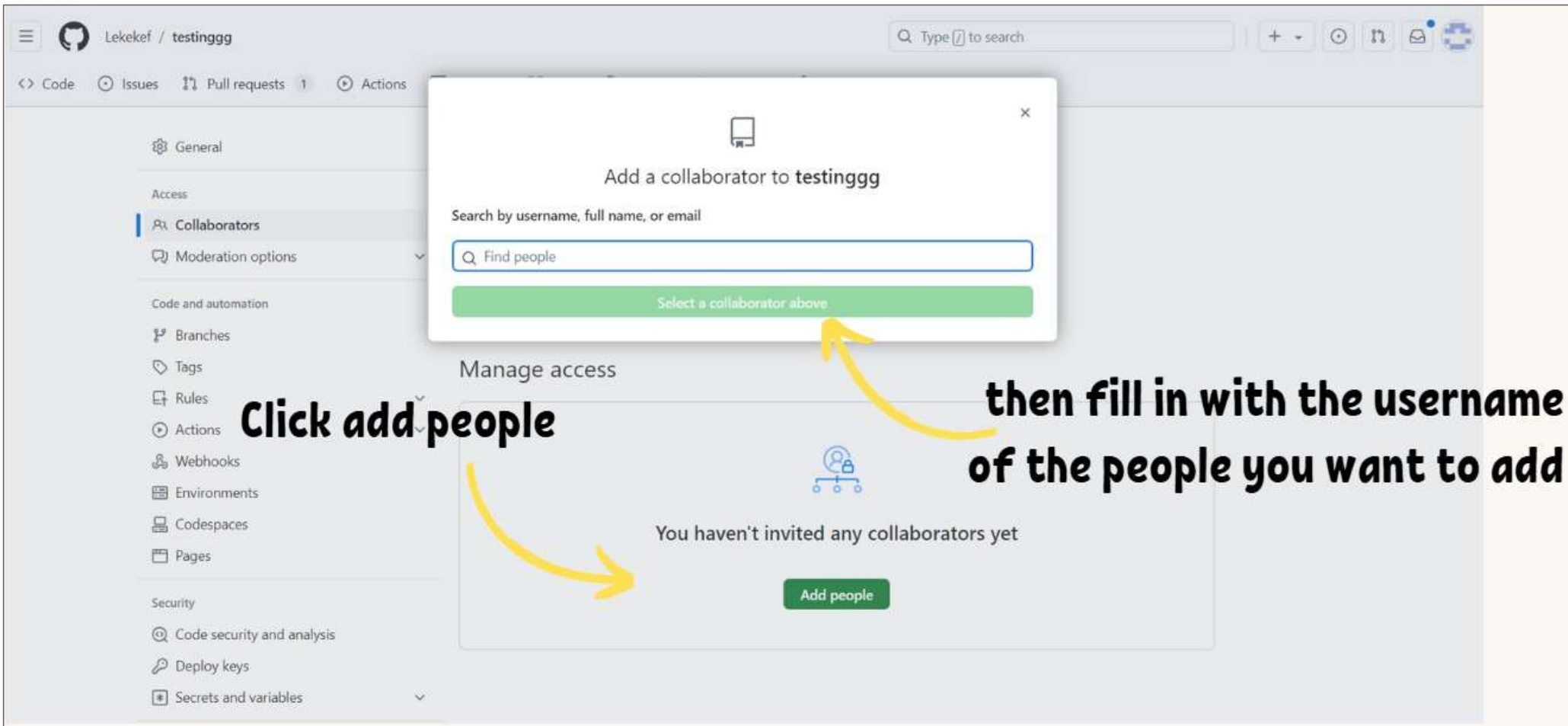
Require contributors to sign off on web-based commits

Default branch

Social preview

<https://github.com/Lekekef/testinggg/settings/access>

# SECURITY ON GitHub: EXAMPLE



Lekekef / testinggg

Code Issues Pull requests 1 Actions

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Code security and analysis

Deploy keys

Secrets and variables

Search Type to search

+

⌚

🔍

✉

🔄

✕

Add a collaborator to testinggg

Search by username, full name, or email

Find people

Select a collaborator above

Manage access

You haven't invited any collaborators yet

Add people

**Click add people**

**then fill in with the username of the people you want to add**

# SECURITY ON GitHub: EXAMPLE

**Once done, we may view the list of people we want to add.**

Manage access

Add people

☐ Select all

Type ▾

Q Cakecap

☐



Cakecap

Awaiting Cakecap's response

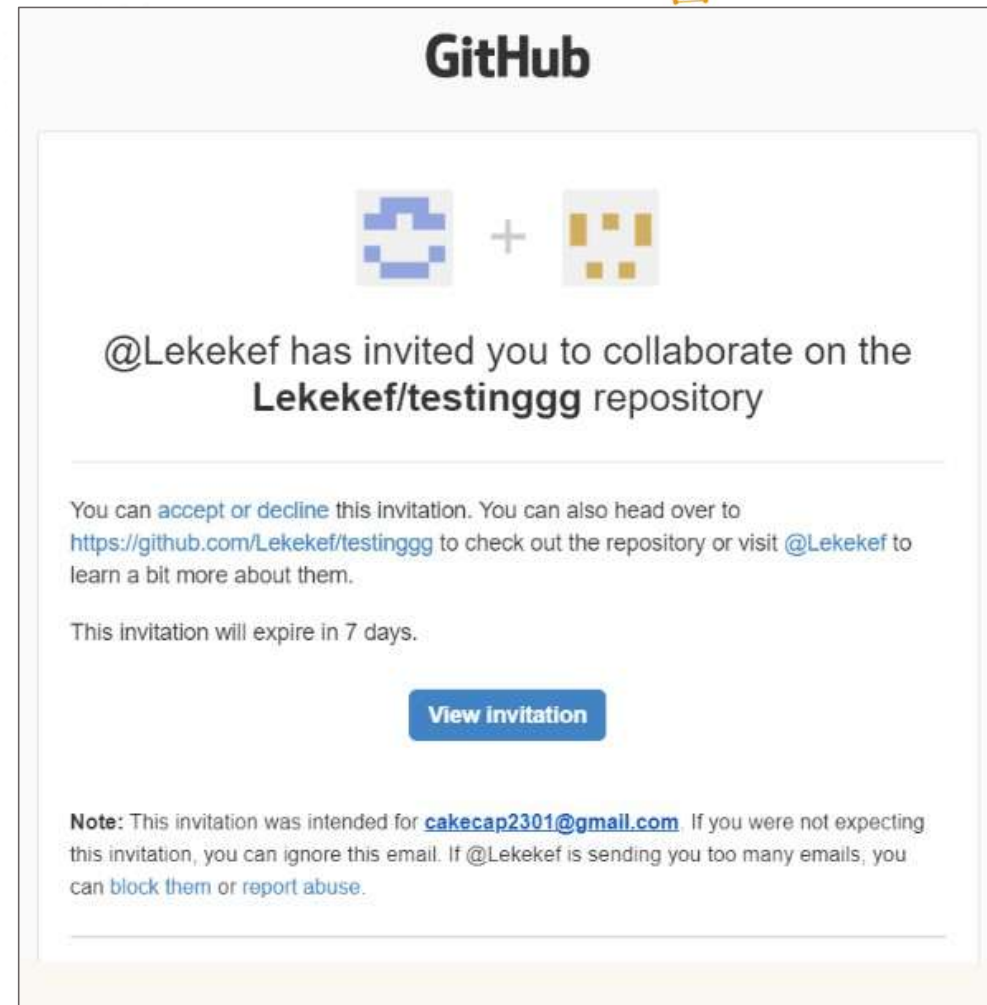
Pending Invite 



**But the invite is actually still pending, So, we need to wait until they accept it via email.**

# SECURITY ON GitHub: EXAMPLE

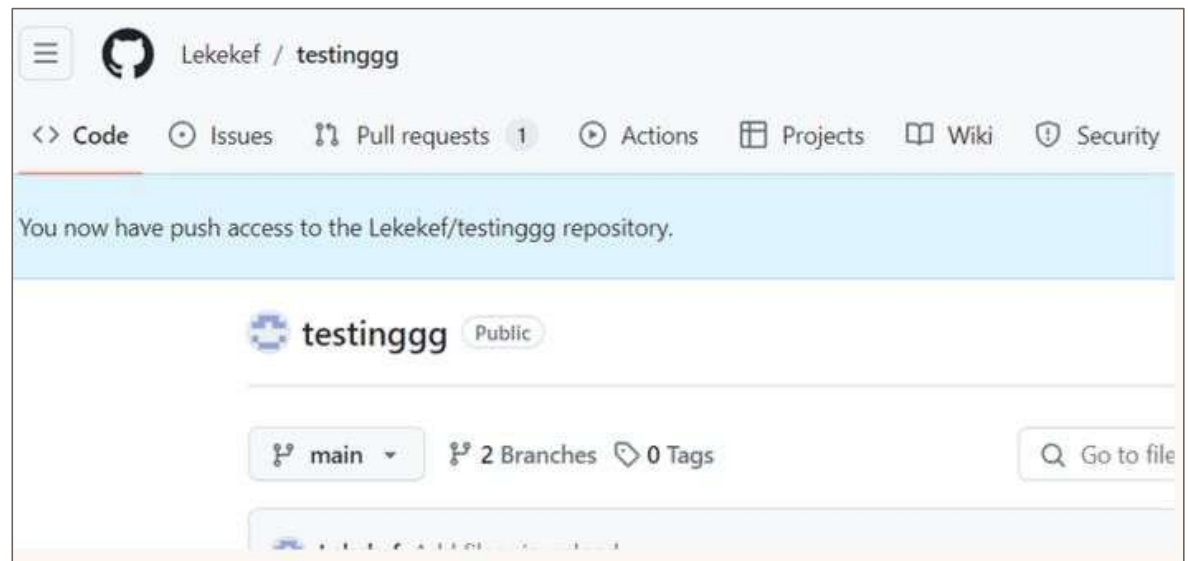
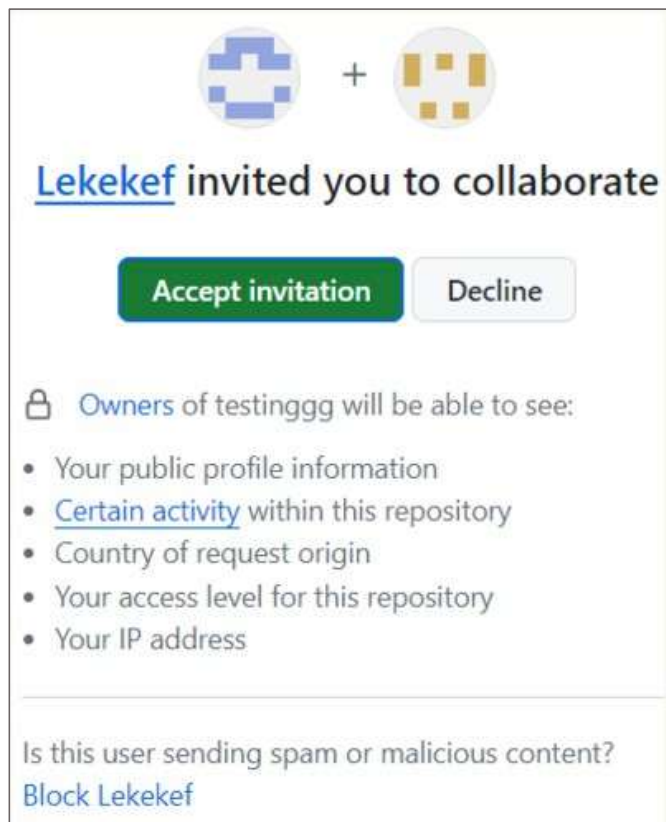
The invited people should get the invitation via email. Click on the “View invitation” to proceed.



# SECURITY ON GitHub: EXAMPLE



Once the invitation is accepted, we may see a notification pop-up indicating the other user has been successfully added



# SETTING UP GitHub IN JETSON



## Installing Git

- Install Git on your Jetson Orin:

*sudo apt update*

*sudo apt install git*

## Configuring Git

- Set up your Git username and email:

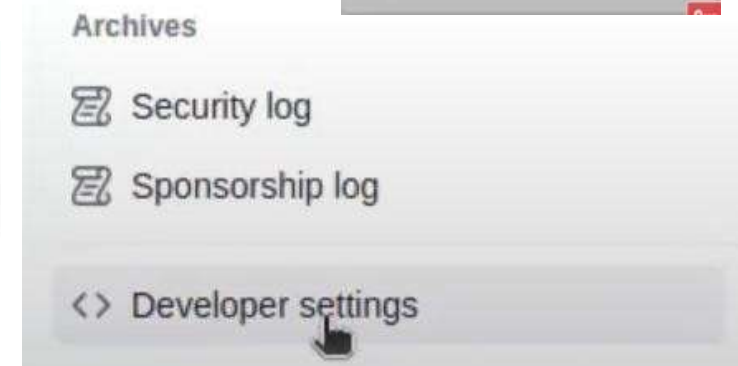
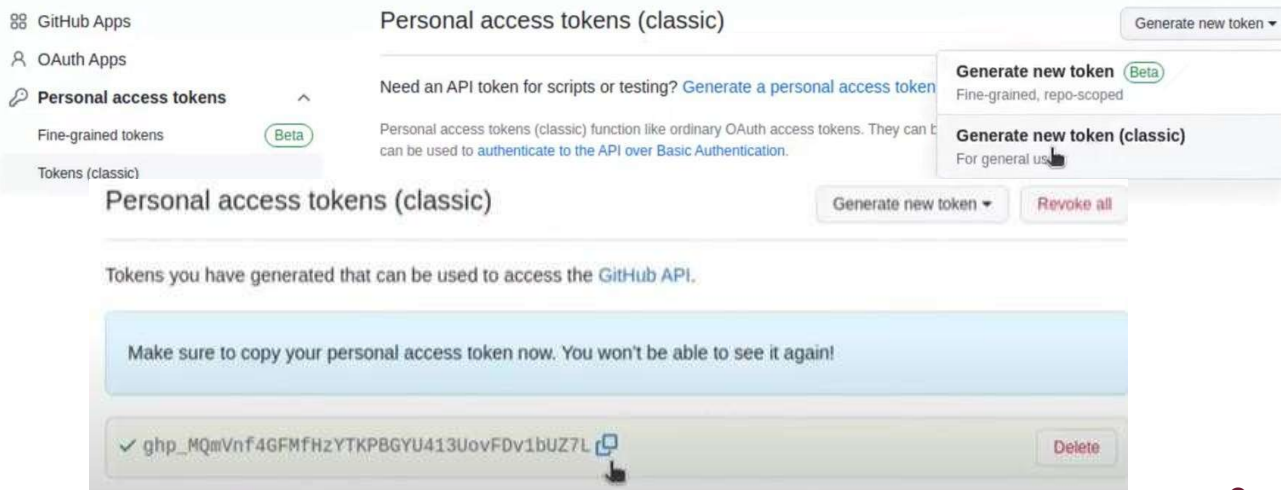
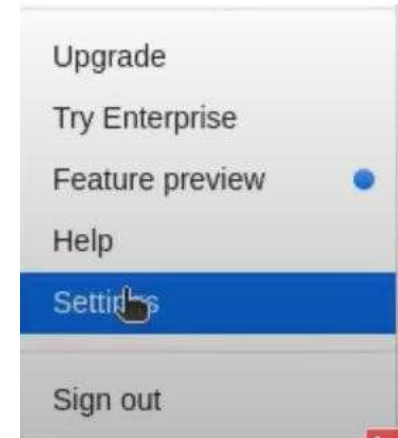
*git config --global user.name "Your Name"*

*git config --global user.email "your.email@example.com"*



# ACCESSING GIT

1. From a GIT client application, you authenticate to the GitHub server by using your **authenticate username** and a **Personal Access Token (PAT)**.
2. Select your profile icon in the top corner, select setting.
3. Then in the left panel select “Developer setting“
4. Click “Personal access tokens” , then generate new token (classic)
5. Fill in the note and select all the scope in generation and generate the token.
6. Copy the token (remember this only will show once)





# JETSON TO AND FROM GitHub



- On the GitHub select your project, select Code on the right and then select the clone URL

- Use the 'git clone' command followed by the repository URL.

*git clone <https://GitHub.com/username/repository.git>*

- After the cloning process completes, you can navigate into the repository's directory:

*Cd repository*

- Adding Files to Git:

*echo "print('Hello, Git!')" > hello\_git.py*

*git add hello\_git.py*

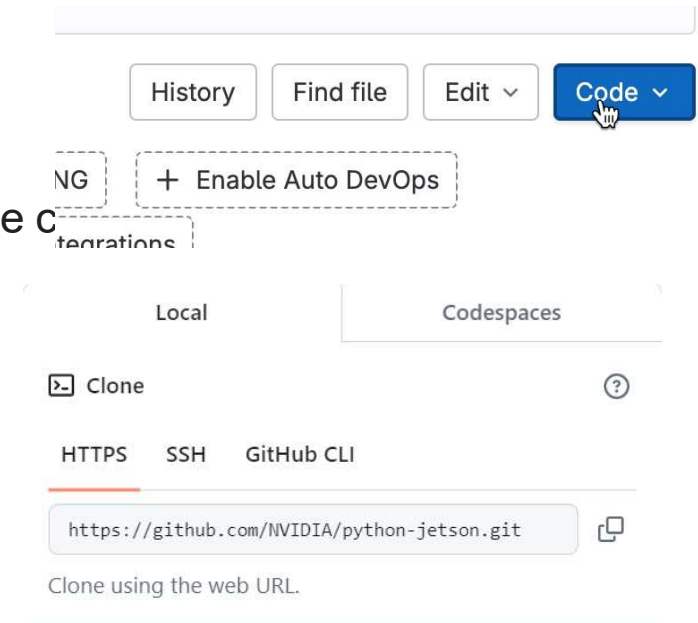
*git commit -m "Initial commit"*

- Pushing to a Remote Repository:

*git status*

*git push origin main*

- Use the token from last slide with your GitHub username.



```
codebind@codebind:~/workspace/hello_test$ git push origin main
Username for 'https://github.com': pknowledge
Password for 'https://pknowledge@github.com':
```

# CONCLUSION



## 1. Professional Development

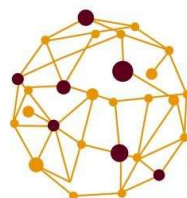
- Continue using GitHub to build a professional portfolio and engage in open-source projects.

## 2. Encouragement to Explore Further

- Apply what you've learned by working on personal or group projects.
- Seek Help and Collaborate: Don't hesitate to seek help from the GitHub community and collaborate with peers.
- Stay Curious: Always be curious and explore new features and best practices.

## 3. Empowerment Through Knowledge

- Understanding and effectively using GitHub will empower you to manage your projects better, collaborate efficiently, and continuously learn and grow as a developer.



**CAIRO**  
UTM

# THANK YOU

 Centre for Artificial Intelligence and Robotics

 [research.utm.my/cairo/](https://research.utm.my/cairo/)

 [cairo@utm.my](mailto:cairo@utm.my) / trainer's email